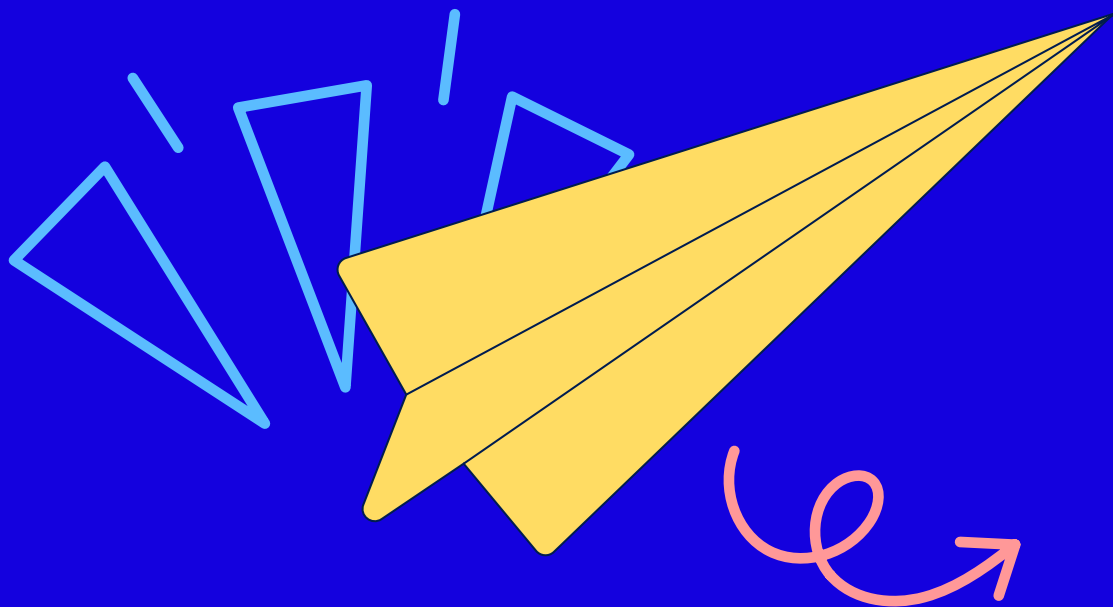




JavaScript Avanzado (1.ª parte)

Frontend



JavaScript Avanzado (1.ª parte)

En el incesante avance del **desarrollo web**, dominar técnicas avanzadas y metodologías emergentes se convierte en una necesidad para aquellos que aspiran a mantenerse a la vanguardia. JavaScript, siendo el alma y el corazón de muchas aplicaciones web modernas, ha evolucionado, presentando paradigmas y características más sofisticadas que simplemente no pueden ser ignoradas por los desarrolladores experimentados.

En este fastbook, vamos más allá de lo básico y **nos adentramos en los aspectos avanzados de JavaScript**. Entenderemos cómo JavaScript procesa nuestro código, viendo en detalle el **event loop**, nos adentraremos en la programación asíncrona y descubriremos técnicas tan modernas como el manejo de promesas y el uso de *frameworks* y bibliotecas contemporáneas. En definitiva, este tema está diseñado para quienes ya poseen conocimientos previos (los fundamentales) de JavaScript y buscan ampliar y perfeccionar sus habilidades, sumergiéndose en las profundidades de lo que este lenguaje puede ofrecer.

Con este fastbook como compañero, estarás equipado para enfrentar los desafíos más intrincados del desarrollo web moderno. Al dominar estos conceptos avanzados, no solo te destacarás en el campo del desarrollo, sino que también estarás listo para innovar.

Autor: Robert Da Corte

 Callbacks Event loop Closures Gestión de errores Promesas y async/await This Modularización ¿Qué hemos aprendido?

Callbacks

Son un concepto fundamental en JavaScript y en muchos otros lenguajes de programación, particularmente en la programación asíncrona.

Es simplemente una función que se pasa a otra función como un argumento, y que se espera que se ejecute después de que la función principal complete su ejecución.

Esto se usa comúnmente en operaciones que llevan tiempo, como solicitudes a servidores, lecturas de archivos o temporizadores.

Las operaciones que no se completan inmediatamente, como las lecturas de archivos en Node.js o las solicitudes AJAX, suelen aceptar funciones de callback para manejar los datos una vez disponibles.



```
// Callback

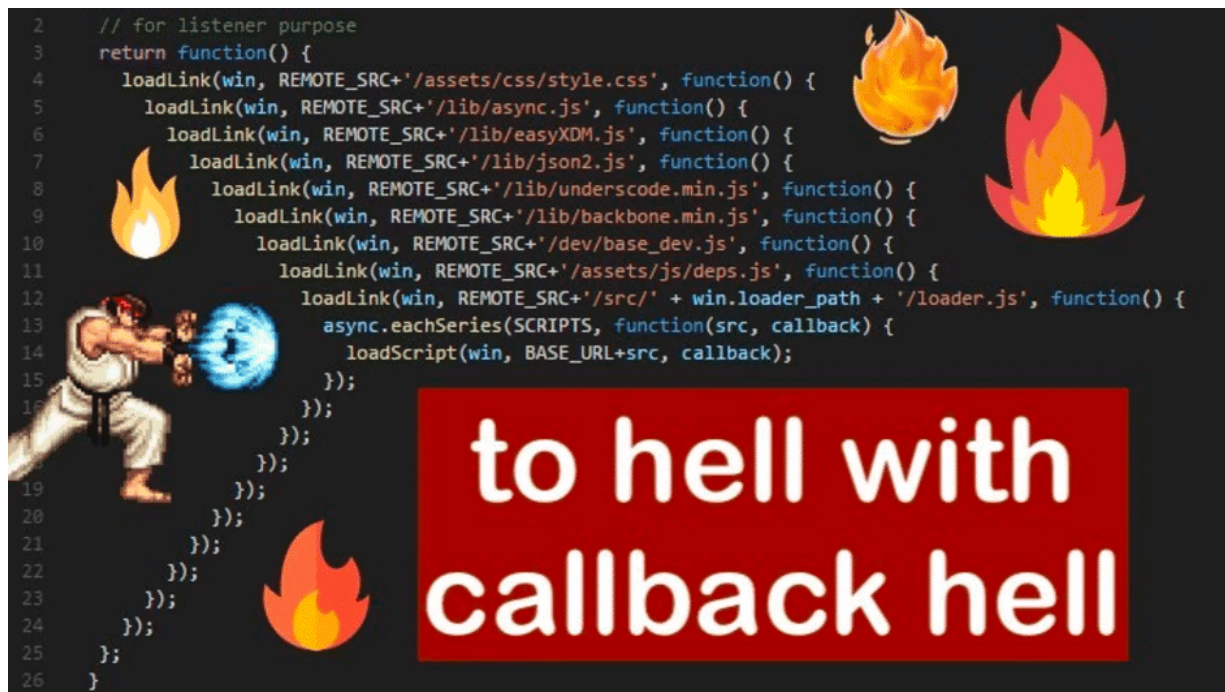
// Ejemplo de callback en Navegador

window.addEventListener('load', function () {
  console.log('La página se ha cargado');
});

// Ejemplo de callback en Node.js
const fs = require('fs');

fs.readFile('archivo', 'utf8', function (err, datos) {
  if (err) {
    console.error("Ocurrió un error", err);
    return;
  }
  console.log(datos); // Muestra los datos leídos
});
```

A medida que encadenas operaciones asíncronas usando callbacks, puedes terminar con múltiples niveles de anidación, lo que hace que el código sea difícil de leer y mantener. Esto se conoce comúnmente como ***callback hell o pirámide de la perdición***.



Para abordar este problema, podemos utilizar diversas estrategias, como modularizar el código en funciones separadas o usar herramientas como promesas o *async/await*, las cuales estudiaremos más adelante.

Event loop

Es un componente fundamental en la forma en que JavaScript maneja las operaciones asíncronas. A pesar de que JavaScript es un lenguaje de programación de un solo hilo, puede realizar **tareas no bloqueantes**, como las llamadas AJAX, los temporizadores y otras operaciones asíncronas, gracias al event loop y otras características como las callbacks, promesas y `async/await`.

Decir que JavaScript es un lenguaje monohilo (*single-threaded*) significa que, en un momento dado, JavaScript puede procesar una sola tarea o instrucción a la vez. En otras palabras, tiene un solo hilo de ejecución. Esto contrasta con los lenguajes o entornos multihilo (*multi-threaded*), donde se pueden ejecutar múltiples tareas o instrucciones simultáneamente en diferentes hilos.

Un hilo es básicamente una secuencia de instrucciones que puede ser ejecutada por el CPU. En un entorno monohilo, solo hay un hilo, lo que significa que solo una instrucción o tarea se procesa a la vez.

Dado que **JavaScript es monohilo**, si una tarea lleva mucho tiempo en completarse (por ejemplo, un bucle intensivo o una operación compleja), puede bloquear el hilo, lo que significa que otras tareas (como la actualización de la interfaz de usuario) tienen que esperar. Este es el motivo por el cual las operaciones intensivas en JavaScript pueden hacer que una página web parezca que se ha *congelado*.

Para solventar este problema, JavaScript puede realizar tareas de manera asíncrona gracias al *event loop*, la *callback queue* y las *Web APIs* proporcionadas por el navegador. Estas operaciones permiten a JavaScript delegar tareas que toman tiempo sin bloquear el hilo principal y, una vez que estas tareas están completas, sus resultados o callbacks pueden ser procesados.

Para entender cómo funciona el event loop, primero debemos familiarizarnos con algunos de sus componentes:

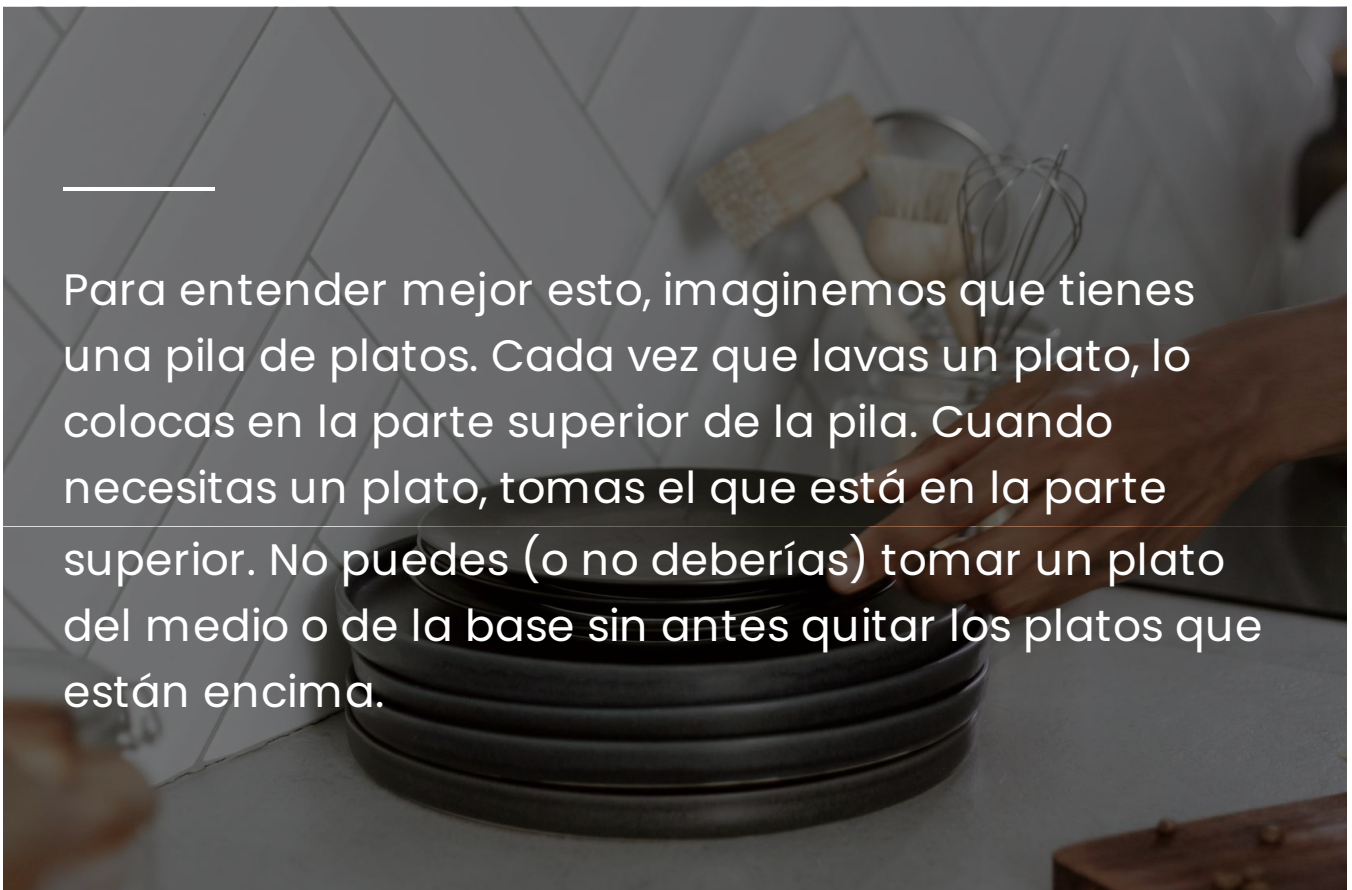
Call stack (pila de llamadas)

1

Call stack (pila de llamadas)

Es donde se almacenan las funciones que se están ejecutando actualmente. Si una función A llama a una función B, entonces B se coloca encima de A en la pila.

Para entender mejor esto, imaginemos que tienes una pila de platos. Cada vez que lavas un plato, lo colocas en la parte superior de la pila. Cuando necesitas un plato, tomas el que está en la parte superior. No puedes (o no deberías) tomar un plato del medio o de la base sin antes quitar los platos que están encima.



Ahora, relacionemos esta analogía con el call stack...

Colocar un plato en la pila —

Esto es similar a cuando se llama a una función en JavaScript. Cada vez que se llama a una función, se coloca en la parte superior del *call stack*.

Tomar un plato de la pila —

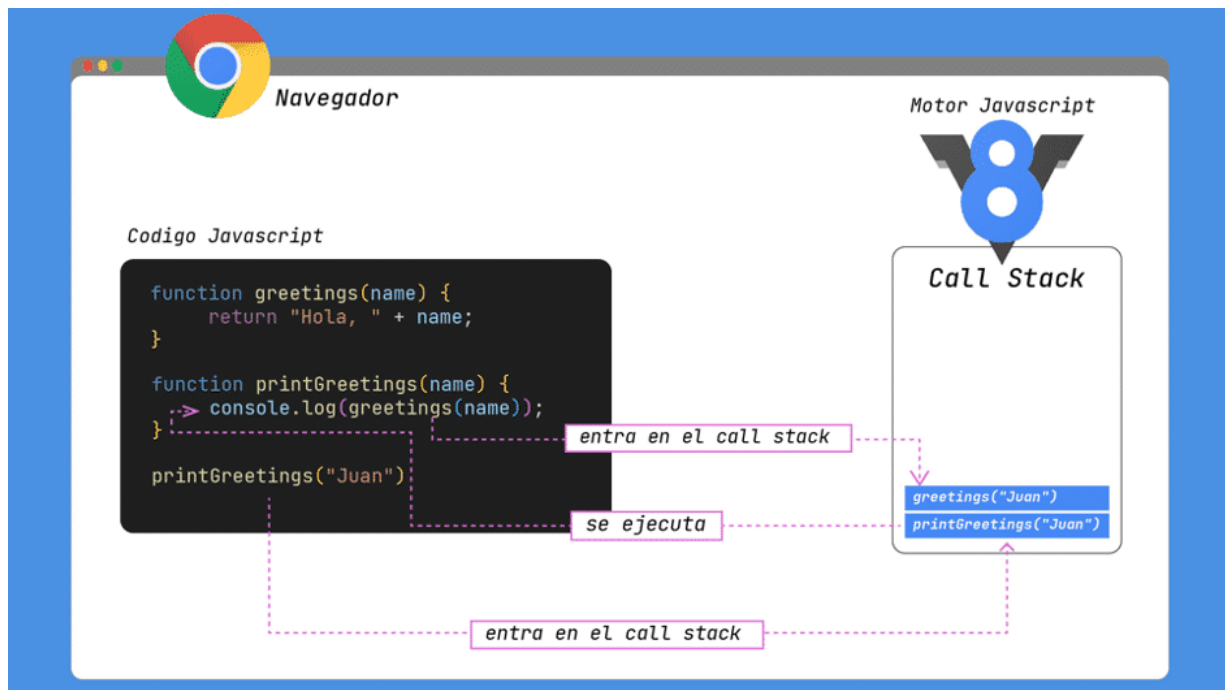
Esto es similar a cuando una función ha terminado su ejecución y regresa un valor o simplemente termina. La función se retira del *call stack*, al igual que tomarías un plato de la parte superior de la pila.

Pila de platos llena —

Si sigues colocando platos en la pila sin parar, eventualmente te quedarás sin espacio y los platos podrían caerse. En el mundo de JavaScript, esto es similar a una situación de *stack overflow*, donde hay demasiadas llamadas a funciones y el *call stack* se llena. Esto puede suceder, por ejemplo, con una recursión mal diseñada donde una función sigue llamándose a sí misma indefinidamente.

Al igual que con los platos, la primera función que entra en el call stack (la que está en la base) será la última en salir, y la última función que entra (la que está en la parte superior) será la primera en salir. Si tienes una función A que llama a una función B, B se coloca encima de A en el call stack. B debe completarse y retirarse del call stack antes de que A pueda completarse.

Aquí tenemos una representación del comportamiento del call stack.

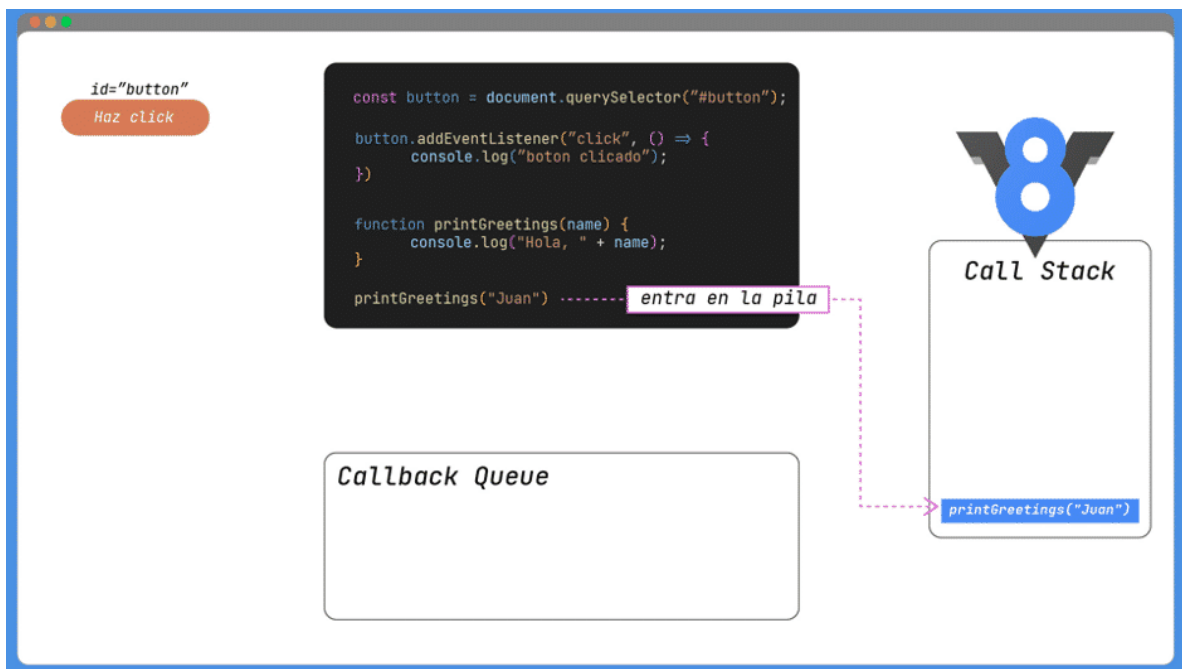


2

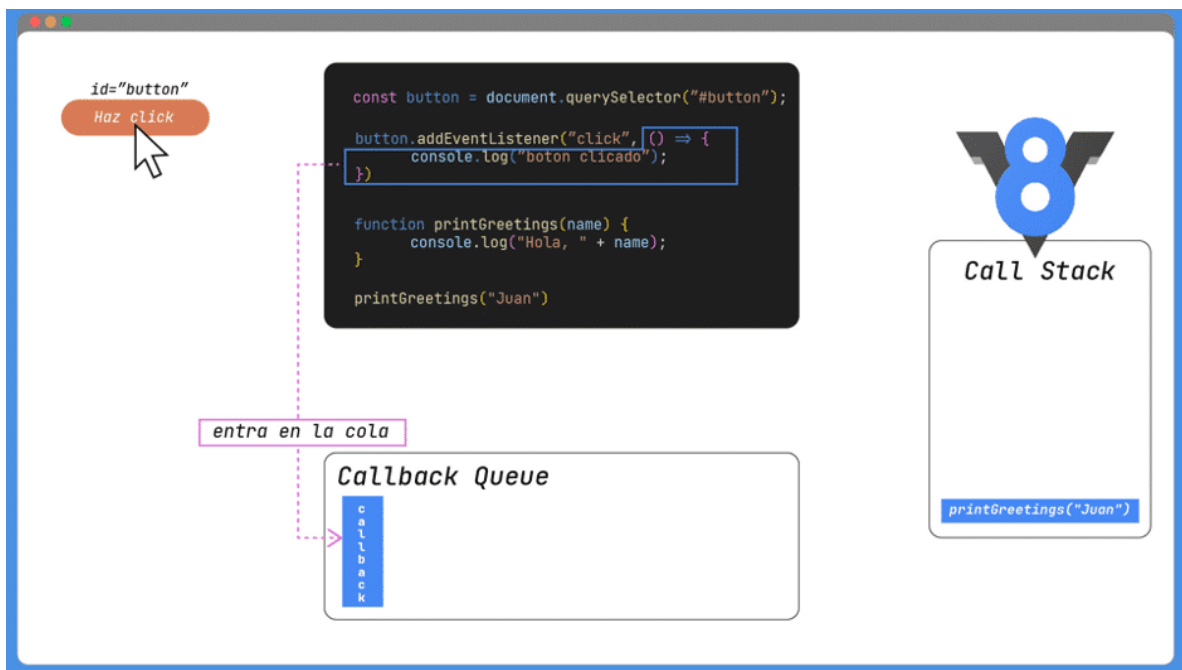
Callback queue (cola de llamadas)

Cuando una operación asíncrona se completa (por ejemplo, un temporizador que se agota o una llamada a servidor que se resuelve), su callback se coloca en esta cola y están esperando ser ejecutados en el call stack.

Imagina que tienes una página web con un botón. En el código JavaScript que se encuentra en esta web tenemos que, cuando se hace clic en el botón, se muestra un mensaje en la consola, pero también tenemos una función que se ejecuta al final del código. Esta ejecución del código hace que la función entre en el call stack como hemos visto en ejemplos anteriores.



Supongamos que ahora un usuario ha clicado nuestro botón, pero aún no se ha terminado de ejecutar la función que está en call stack. Lo que pasa de manera intermitente es que la acción que queremos ejecutar cuando el usuario hace clic, entra en una 'fila' a la espera de que el event loop la meta en el call stack.



Una vez el call stack queda limpio, el event loop pasa por la *callback queue* y mueve, de una en una y en orden de llegada, cada una de las funciones que están a la espera de ser ejecutadas al call stack.



Una vez que el callback ha sido ejecutado, se retira del call stack, y el event loop continúa monitoreando el call stack y el callback queue para futuras tareas.

Se refiere a un conjunto de APIs o funcionalidades proporcionadas por el entorno de ejecución (por ejemplo, el navegador web) que permiten a JavaScript realizar operaciones que van más allá de sus capacidades intrínsecas, como operaciones asíncronas, la manipulación del DOM, interacciones con el sistema de archivos (en el caso de Node.js), y más.

Estas Web APIs no son parte del lenguaje JavaScript en sí, sino que son proporcionadas por el entorno en el que se ejecuta JavaScript. Cuando se invoca una de estas funciones, la operación se maneja fuera del *thread* principal de JavaScript, permitiendo que el lenguaje continúe ejecutando otras tareas sin ser bloqueado por la operación de la Web API.

Supongamos que tienes el siguiente código:



```
// Web API: setTimeout

console.log("Inicio");

setTimeout(function() {
  console.log("Callback de setTimeout");
}, 2000);

console.log("Fin");
```

En este caso, el motor de JavaScript ejecuta el primer *console.log*, y verás "Inicio" en la consola. Luego, el motor de JavaScript encuentra la función *setTimeout*, que es una Web API proporcionada por el navegador.

En lugar de bloquear la ejecución y esperar 2000 milisegundos (2 segundos), el navegador toma el control de esta operación. La función que recibe como argumento *setTimeout*, también conocido como **callback**, se pasa junto con el tiempo de espera (2000 ms) al navegador.

Mientras el navegador está esperando para ejecutar el callback después de los 2000 ms, el motor de JavaScript continúa ejecutando el código restante. Por lo tanto, verás "Fin" en la consola.

Una vez que han pasado los 2000 ms, el navegador coloca el callback de *setTimeout* en la **callback queue**. El event loop verifica si el **call stack** está vacío. Una vez que está vacío, el callback se mueve al Call Stack y se ejecuta, mostrando "Callback de *setTimeout*" en la consola.

En este ejemplo, la Web API (en este caso, la función *setTimeout* proporcionada por el navegador) permite a JavaScript realizar una operación asíncrona sin bloquear el hilo principal. Es importante entender que estas Web APIs son proporcionadas por el entorno (navegador, Node.js, etc.) y no por el lenguaje JavaScript en sí.

Aquí te dejo algunas de las webs API más utilizadas y las funcionalidades que ofrecen:

- **DOM** permite manipular e interactuar con el contenido de una página web. Métodos y propiedades como *getElementById*, *querySelector*, *appendChild*, etc.
- **Fetch API** proporciona una interfaz para obtener recursos a través de la red. Se utiliza para hacer solicitudes HTTP y reemplaza al antiguo objeto *XMLHttpRequest*.
- **Geolocation API** permite obtener la ubicación geográfica del usuario (con su permiso).
- **Web Storage API** proporciona mecanismos para almacenar datos en el navegador del cliente. Incluye *localStorage* y *sessionStorage*.
- **File API** permite interactuar con archivos en el sistema del cliente. Se utiliza, por ejemplo, para leer el contenido de un archivo que un usuario ha seleccionado a través de un input de tipo "file".
- **Canvas API** permite dibujar gráficos y animaciones en un elemento `<canvas>`.
- **Web Audio API** proporciona una manera de procesar y manipular *streams* de audio en aplicaciones web.
- **WebSocket API** permite establecer conexiones bidireccionales y en tiempo real con servidores.
- **Notifications API** permite mostrar notificaciones a los usuarios.

- **History API** ofrece la posibilidad de interactuar con el historial de navegación del navegador. Métodos como *pushState* y *replaceState* permiten modificar el historial sin recargar la página.
- **Service Workers API** permite ejecutar *scripts* en segundo plano, independientemente de la página web, lo que es fundamental para características como las *Progressive Web Apps* (PWAs), la caché y las notificaciones *push*.
- **IndexedDB API** es una base de datos en el navegador para almacenar datos estructurados y realizar operaciones de alta performance.
- **WebRTC (Web Real-Time Communications) API** permite la comunicación en tiempo real entre navegadores, como videollamadas y chats.
- **Performance API** proporciona acceso a la información sobre el rendimiento de la página web.
- **Navigation Timing API** proporciona información detallada sobre el rendimiento de navegación y carga de la página.



Recuerda: a medida que vayas evolucionando como desarrollador, tendrás que utilizar otras WEB APIs no mencionadas, no te agobies, en esta área vamos aprendiendo según vamos resolviendo diferentes problemas que se nos presentan.

No tienes por qué saber en detalle qué hace cada Web API, ya que siempre te podrás documentar sobre cómo se trabaja con ella en la web oficial de [Mozilla](#).

Closures

Las closures en JavaScript son funciones que hacen referencia a variables independientes.

La función definida en la closure *recuerda* el entorno en el que fue creada, incluso después de que la función externa haya terminado de ejecutarse.

```
// Closure

function greetingFactory(greeting) {
  return function (name) {
    // La función interna tiene acceso a la variable "greeting" de la función externa
    // cuando es retornada por la función externa,
    // la función interna se convierte en un closure
    // y "greeting" se convierte en una variable privada para la función interna
    console.log(`¡${greeting}, ${name}!`); ¡Hola, Pepe!, ¡Adiós, Pepe!
  };
}

const sayHello = greetingFactory('Hola');
const sayGoodbye = greetingFactory('Adiós');

sayHello('Pepe'); // Imprime: "Hola, Pepe!"
sayGoodbye('Pepe'); // Imprime: "Adiós, John!"
```

En este ejemplo, ***greetingFactory*** es una función que devuelve otra función. La función devuelta, aunque es invocada fuera del contexto de ***greetingFactory***, todavía tiene acceso a la variable *greeting* de su ámbito de creación.

Se usan en el día a día porque permiten emular algo parecido a variables privadas. Es decir, puedes tener valores que no pueden ser modificados directamente desde fuera de la función, pero sí a través de funciones específicas. Más adelante en el fastbook, veremos qué utilidad nos aportan.

Gestión de errores

La gestión de errores es un pilar fundamental en cualquier lenguaje de programación. En JavaScript, la estructura *try/catch* brinda una forma de identificar y responder a errores durante la ejecución del programa.

```
// Error handling

try {
  // Al intentar acceder a una variable que no existe, se genera un error
  console.log(tryingToAccessUndefinedVariable);
  const tryToAccessUndefinedVariable = 1;
} catch (error) {
  // El error se captura en el catch
  [ReferenceError: tryingToAccessUndefinedVariable is not defined]
  console.log(error); [ReferenceError: tryingToAccessUndefinedVariable is not defined]
}
```

El bloque *try* contiene el código que puede generar errores. Si surge un error, la ejecución se desvía al bloque *catch*, donde se puede gestionar dicho error. Tras *try* y *catch*, es posible añadir un bloque *finally*. El código dentro de *finally* se ejecutará después de *try* y *catch*, independientemente del resultado.

```
// Error handling

try {
  console.log(tryingToAccessAnUndefinedVariable);
  const tryingToAccessAnUndefinedVariable = 1;
} catch (error) {
  console.log(error.name); ReferenceError
  console.log(error.message); Cannot access 'tryingToAccessAnUndefinedVariable' before initialization
} finally {
  // Siempre se ejecuta
  // aqui podemos ocultar un spinner de carga,
  // cerrar conexiones a bases de datos, etc.

  console.log('Ocultar spinner de carga'); Ocultar spinner de carga
}
```

El objeto error

Dentro del bloque *catch*, se tiene acceso a un objeto error que detalla el error ocurrido. Comúnmente, este objeto tiene al menos dos **propiedades clave**:

- *error.name*: el tipo de error (por ejemplo, *ReferenceError*, *TypeError*, etc.).
- *error.message*: un mensaje que describe el error.

```
// Error handling

try {
  // Al intentar acceder a una variable que no existe, se genera un error
  console.log(tryingToAccessUndefinedVariable);
  const tryToAccessUndefinedVariable = 1;
} catch (error) {
  // El error se captura en el catch
  console.log(error.name); ReferenceError
  console.log(error.message); tryingToAccessUndefinedVariable is not defined
}
```

Además del genérico Error, JavaScript tiene diversos **objetos de error incorporados**:

- **ReferenceError**: se produce al referenciar una variable inexistente.
- **TypeError**: surge al usar un valor de manera incorrecta.
- **SyntaxError**: indica un error de sintaxis en el código.
- **RangeError**: se produce cuando un valor está fuera del rango admitido.
- **EvalError**: relacionado con la función `eval()`.
- **URIError**: relacionado con funciones de manipulación de URIs.

También es posible generar errores propios con la palabra clave `throw`. Esto es útil para indicar un error basado en una condición específica.

```
// Error handling

function hasToBePositive(number) {
  if (number < 0) {
    // Lanzamos un error con el mensaje que queramos
    throw new Error('The number has to be positive');
  }
}

try {
  // Llamamos a la función con un número negativo
  hasToBePositive(-1);
  console.log('The number is positive'); // Esta línea no se ejecuta
} catch (error) {
  console.log(error.name); Error
  console.log(error.message); The number has to be positive
}
```

Promesas y `async/await`

Las promesas son un mecanismo poderoso para la gestión de operaciones asíncronas. Permiten un enfoque más estructurado y comprensible en comparación con los callbacks tradicionales. Es un objeto que representa la terminación o el fallo eventual de una operación asíncrona. Esencialmente, es una devolución de llamada para el futuro.

Las promesas tienen **tres estados** posibles:

- 1** **Pendiente** (*Pending*): el estado inicial, no se ha cumplido ni rechazado.
- 2** **Cumplida** (*Fulfilled*): significa que la operación se completó con éxito.
- 3** **Rechazada** (*Rejected*): significa que la operación falló.

Crear una promesa

Una promesa se crea usando el constructor *Promise*. Este constructor toma una función como argumento, la cual, a su vez, recibe dos argumentos: *resolve* y *reject*.

- **Resolve** es una función que se llama cuando la operación asíncrona se completa con éxito.
- **Reject** es una función que se llama cuando la operación asíncrona falla.

```
// Promises

const asynchronousFunction = new Promise((resolve, reject) => {
  // Simulamos una operación asíncrona usando setTimeout
  const exito = true;

  setTimeout(() => {
    if (exito) {
      resolve('Operación exitosa'); // resolve se ejecuta si la promesa se cumple
    } else {
      reject('Error en la operación'); // reject se ejecuta si la promesa no se cumple
    }
  }, 2000);
});
```

En este ejemplo, después de 2 segundos, la promesa se resuelve con éxito.

Para manejar los resultados de una promesa, utilizamos `.then()`, para el caso de éxito, y `.catch()` para errores. Siguiendo el ejemplo anterior, veremos que la promesa se resolverá de manera exitosa.

```
// Promises

const asynchronousFunction = new Promise((resolve, reject) => {
  // Simulamos una operación asíncrona usando setTimeout
  // En este caso, la operación se cumple (exito = true)
  const exito = true; // Como exito es true, la promesa se cumple

  setTimeout(() => {
    if (exito) {
      resolve('Operación exitosa'); // resolve se ejecuta si la promesa se cumple
    } else {
      reject('Error en la operación'); // reject se ejecuta si la promesa no se cumple
    }
  }, 2000);
});

asynchronousFunction.then(resultado => {
  console.log(resultado); // Operación exitosa
});
```

Cambiando el valor de la constante **exito** a *false*, haremos que la promesa falle y podremos capturar el resultado con **.catch()**.

```
// Promises

const asynchronousFunction = new Promise((resolve, reject) => {
  // Simulamos una operación asíncrona usando setTimeout

  const exito = false; // Como exito es false, la promesa no se cumple

  setTimeout(() => {
    if (exito) {
      resolve('Operación exitosa'); // resolve se ejecuta si la promesa se cumple
    } else {
      reject('Error en la operación'); // reject se ejecuta si la promesa no se cumple
    }
  }, 2000);
});

asynchronousFunction.then(resultado => {
  console.log(resultado);
}).catch(error => {
  console.log(error); Error en la operación
});
```

Al igual que el bloque **try/catch** las promesas también disponen un tercer método llamado **.finally()** que se ejecuta siempre, indiferentemente de si la promesa se resuelve o falla.

```
asynchronousFunction.then(resultado => {
  console.log(resultado);
}).catch(error => {
  console.log(error); Error en la operación
}).finally(() => {
  console.log("Oculta el spinner"); Oculta el spinner
});
```

Una ventaja de las promesas es que pueden ser encadenadas, es decir, una operación asíncrona puede seguir a otra y así sucesivamente, tal y como vemos en la siguiente imagen.

```
// Promises

// Primero definimos las promesas que queremos ejecutar

const serverRequest = new Promise((resolve, reject) => {
  const exito = true; // Como exito es false, la promesa no se cumple
  setTimeout(() => {
    if (exito) {
      const data = [{ id: 1, amount: 200 }];
      resolve(data); // resolve se ejecuta si la promesa se cumple
    } else {
      const error = new Error('Algo salió mal');
      reject(error); // reject se ejecuta si la promesa no se cumple
    }
  }, 2000);
});

const processData = data => {
  return new Promise(resolve => {
    const result = data.map(element => element.amount * 2);
    resolve(result);
  });
};
```

Y la forma de encadenarlas sería de la manera que muestro a continuación.

```
// Podemos encadenar promesas
serverRequest.then( data => {
  processData(data).then( data => {
    console.log(data); [ 400 ]
  }).catch(error => {
    console.log(error);
  });
}).catch(error => {
  console.log(error);
});
```

Métodos útiles

Adicional a **reject** y **resolve**, el objeto **Promise** posee una serie de **métodos adicionales** útiles para trabajar con operaciones asíncronas.

- **Promise.all()**: acepta un array de promesas y devuelve una nueva promesa que se cumple, cuando todas las promesas en el array se han cumplido o cuando alguna de ellas se rechaza.
- **Promise.race()**: acepta un array de promesas y devuelve una nueva promesa que se cumple o rechaza tan pronto como una de las promesas en el array se cumple o se rechaza.

- ***Promise.any()***: retorna una promesa que se resuelve tan pronto como una de las promesas del iterable se cumpla. Si todas las promesas son rechazadas, entonces la promesa devuelta es rechazada con un ***AggregateError***, un nuevo tipo de error que agrupa esos errores individuales.
- ***Promise.allSettled()***: retorna una promesa que se resuelve cuando todas las promesas en el iterable han sido resueltas, ya sea que se hayan cumplido o rechazado. Es útil cuando necesitas asegurarte de que todas las promesas se hayan completado, independientemente de su resultado.



¡Atención! Te recomiendo siempre que visites la documentación oficial de [Mozilla](#) en donde podrás conocer funcionalidades adicionales.

Async / Await

Es una adición introducida en ES6 que facilita el trabajo con operaciones asíncronas y promesas. Esencialmente, permite escribir código asíncrono como si fuera síncrono, logrando que sea más legible y manejable. Cuando colocas la palabra reservada ***async*** antes de una función la convierte en una función asíncrona, lo que significa que siempre devolverá una promesa. Incluso si retornas un valor que no es una promesa, JavaScript automáticamente lo envolverá en una promesa resuelta.

```
// Async Await

async function getValue() {
  return "¡Hola Async!";
}

// JavaScript interpreta la función anterior como si fuera asíncrona,
// incluso si no devuelve una promesa explícitamente.

getValue().then(console.log); // ¡Hola Async!
```

Dentro de una función **async**, puedes usar la palabra clave **await** para esperar que una promesa se resuelva. Esto pausa la ejecución de la función hasta que la promesa se cumpla, permitiéndote asignar su valor resuelto a una variable.

```
// Async Await

const serverRequest = new Promise((resolve, reject) => {
  const exito = true;
  setTimeout(() => {
    if (exito) {
      const data = [{ id: 1, amount: 200 }];
      resolve(data);
    } else {
      const error = new Error('Algo salió mal');
      reject(error);
    }
  }, 2000);
});

async function fetchData() {
  try {
    const data = await serverRequest; // Espera a que se resuelva la promesa
    console.log(data); [ { id: 1, amount: 200 } ]
  } catch (error) {
    console.log(error);
  }
}

fetchData();
```

Recuerda: solo puedes usar *await* dentro de funciones *async*.

This

Aunque parece simple, su comportamiento cambia según el contexto en el que se encuentra. El valor de **this**, en el contexto de una función, es una referencia al objeto en el que opera dicha función. ¿Pero, qué objeto es ese? Bueno, eso depende de cómo se invoca la función.

En los siguientes ejemplos veremos cómo *this* se referirá a algo diferente dependiendo del contexto en el que se ejecute.

Contexto global

Si llamamos a una función de forma regular (no como método de un objeto o constructor), **this** se referirá al objeto global. En navegadores, ese objeto global es *window*. En entornos Node.js es *global*.

```
// This

function testThis() {
  console.log(this);
}

testThis(); // En navegadores, mostrará el objeto `window`.
```

Métodos de objetos

Dentro de un método de objeto, **this** se refiere al objeto que contiene el método.

```
// This

const obj = {
  name: "Pepe",
  sayHello: function() {
    console.log("Hola " + this.name);
  }
}

obj.sayHello(); // Salida: "Hola Pepe"
```

Constructores y clases

Cuando una función se utiliza como constructor (con el operador **new**), **this** se refiere al nuevo objeto que se está creando.

```
// This

function Person(name) {
  this.name = name;
}

const john = new Person("Juan");
console.log(john.name); // Salida: "Juan"
```

Eventos

En manejadores de eventos, **this** suele referirse al elemento que ha disparado el evento.

```
// This

button.addEventListener('click', function() {
  console.log(this); // Se refiere al elemento button.
});
```

Arrow functions

Las funciones de flecha no tienen su propio **this**. En su lugar, capturan el valor de **this** del entorno circundante.

```
// This

function outerFunction() {
  console.log(this); // Por ejemplo, podría ser `window` o
  cualquier otro objeto.

  const arrowFunction = () => {
    console.log(this); // Tendrá el mismo valor que el `this`
    de `outerFunction`.
  }

  arrowFunction();
}
```

Si bien este es el comportamiento de *this*, mediante métodos como *bind()*, *call()*, y *apply()*, puedes especificar o cambiar el contexto de *this* para una función.

Call

El método `.call()` permite llamar a una función con un objeto específico como valor de *this* y argumentos individuales.

```
// This: Call

const person = {
  fullName: function() {
    return `${this.firstName} ${this.lastName}`;
  }
};

const john = {
  firstName: "Juan",
  lastName: "Lopez"
};

// En este caso call hace la variable john sea el objeto a ser referido cuando la función fullName de person
// intente acceder a this.firstName y this.lastName

console.log(person.fullName.call(john)); // Juan Lopez
```

En el ejemplo anterior, a pesar de que *john* no tiene un método *fullName*, pudimos usar el método *fullName* de *person* y cambiar su contexto de *this* a *john* usando *call()*.

Adicionalmente, también podemos pasar argumentos adicionales después de objeto a ser referido en *call*.



```
// This: Call

const person = {
  greet: function(city, country) {
    return `Hola, mi npmbre es ${this.name} y vivo en ${city}, ${country}.`;
  }
};

const alice = {
  name: "Alicia"
};

console.log(person.greet.call(alice, "Madrid", "España")); // `Hola, mi npmbre es Alicia y vivo Madrid, España`
```

Apply

Al igual que ***call()***, el método [apply\(\)](#) permite invocar una función determinando explícitamente el valor de *this*. Sin embargo, la principal diferencia entre ***call()*** y ***apply()*** radica en cómo pasas los argumentos a la función: mientras que ***call()*** acepta una lista de argumentos, ***apply()*** acepta un solo array de argumentos.

Tomemos el mismo ejemplo usado en *call*.

```
// This: Call

const person = {
  greet: function(city, country) {
    return `Hola, mi nombre es ${this.name} y vivo en ${city}, ${country}.`;
  }
};

const alice = {
  name: "Alicia"
};

console.log(person.greet.apply(alice, ["Madrid", "España"])); // `Hola, mi nombre es Alicia y vivo Madrid, España`
```

Fíjate cómo con *apply()* los argumentos "Madrid" y "España" se pasan dentro de un array.

Bind

Y aquí tienes un ejemplo sencillo de la función [bind](#).

```
// This: Bind

const person = {
  firstName: 'Juan',
  lastName: 'Lopez',
  fullName: function () {
    return `${this.firstName} ${this.lastName}`;
  }
};

const unboundFullName = person.fullName;
console.log(unboundFullName()); // Esto va a devolver "undefined undefined"
// porque `this` ya no se refiere al objeto `person`.

const boundFullName = unboundFullName.bind(person);
console.log(boundFullName()); // Esto devolverá "Juan Lopez"
// porque hemos vinculado `this` al objeto `person`.
```

Modularización

Modularizar significa dividir tu código en piezas más pequeñas y reutilizables (módulos) con el propósito de mantener el código organizado, evitar colisiones de nombres y facilitar la reutilización y el mantenimiento.

Desde ES6 en adelante, JavaScript ha introducido un sistema nativo de módulos que ha sido adoptado en la mayoría de los entornos modernos, como navegadores y Node.js.

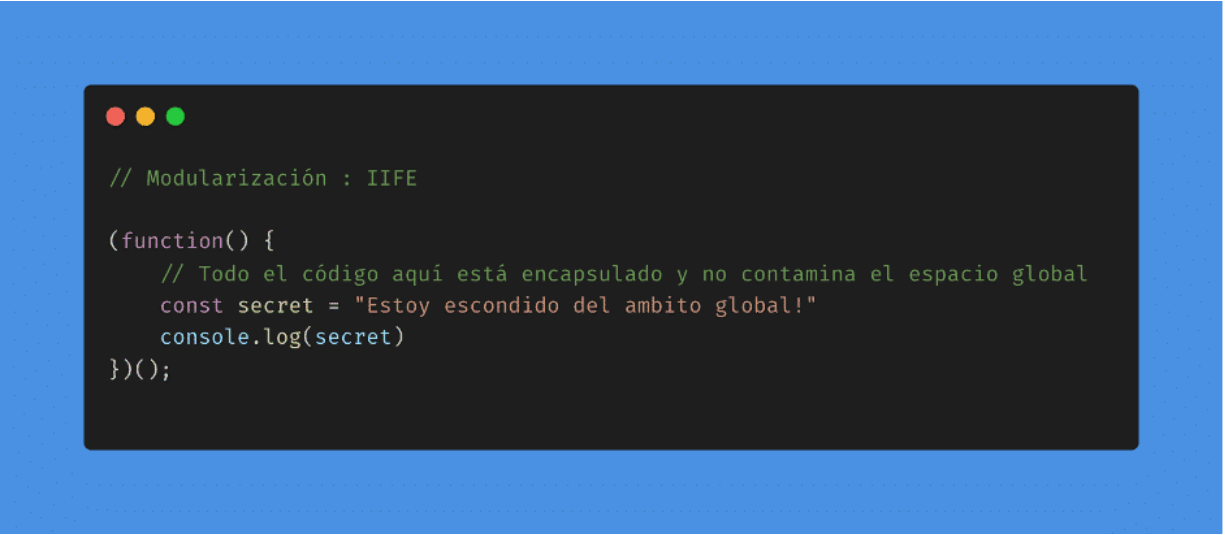
A continuación, exploraremos las diferentes formas que tenemos para modularizar nuestro código.

Encapsulación con IIFE (*Immediately Invoked Function Expressions*)

Es un patrón en JavaScript que se utiliza para ejecutar una función inmediatamente después de que se ha definido y una de las maneras más comunes de encapsular código en JavaScript, especialmente antes de la popularidad y el amplio soporte de los módulos ES6.

IIFE es una función que se ejecuta inmediatamente después de ser definida. Se define como una expresión y se ejecuta de inmediato.

La principal razón para usar un IIFE es para **encapsular y aislar el código**, manteniendo las variables y funciones definidas dentro del IIFE fuera del alcance global. Esto ayuda a prevenir la contaminación del espacio de nombres global y posibles conflictos de nombres.



```
// Modularización : IIFE

(function() {
  // Todo el código aquí está encapsulado y no contamina el espacio global
  const secret = "Estoy escondido del ambito global!"
  console.log(secret)
})();
```

También se pueden pasar argumentos a una IIFE, lo que nos sirve para hacer referencia a objetos globales.



```
// Modularización: IIFE

(function(window, document) {
    // Ahora podemos usar 'window' y 'document' dentro de esta función
})(window, document);
```

Aunque con la introducción de módulos en ES6, la necesidad de IIFE para la encapsulación puede ser menos prominente, sigue siendo una técnica valiosa y ampliamente utilizada, especialmente en scripts que deben ser compatibles con versiones anteriores de JavaScript o en situaciones donde los módulos ES6 no son una opción viable.

ESM

Con ES6 se introdujo una sintaxis nativa para **importar y exportar módulos**. Representa una mejora significativa en la forma de organizar y gestionar el código JavaScript en aplicaciones y bibliotecas más grandes.

Antes de ESM, los desarrolladores se apoyaban en sistemas de módulos como *CommonJS* (utilizado principalmente en Node.js) o *AMD (Asynchronous Module Definition)*, utilizado principalmente en navegadores), sistemas que explicaremos más adelante.

```
// Modularización: ESM

// Exportamos una serie de funciones y propiedades que podemos utilizar
// en diferentes partes de nuestra web / aplicación

// Estas funciones se encuentran en un fichero llamado math.js

export function sum(x,y){
  return x + y;
}

export function multiply = (x, y) => x * y;

export const PI = 3.14159;
```

Y para importar las funciones sería de la siguiente manera, ¡presta atención!

```
// Modularización: ESM

// Para consumir las funciones exportadas en el fichero math.js lo hacemos de la siguiente forma
import { sum, multiply, PI } from './math.js'

console.log(sum(2,3)) // 5

// O también podríamos importarlas todas y meterlas dentro de un objeto al vuelo.
import * as math from './math.js'

console.log(math.multiply(2,2)) // 4
```

CommonJS

Es un sistema de módulos popularmente conocido por su uso en Node.js. Fue uno de los primeros intentos de estandarizar cómo se podría **estructurar y modularizar código en JavaScript** fuera de los navegadores. Antes de la introducción de ES6 modules (ESM), CommonJS proporcionó una solución fundamental para resolver problemas de dependencias y estructuración de código en proyectos más grandes de JavaScript.

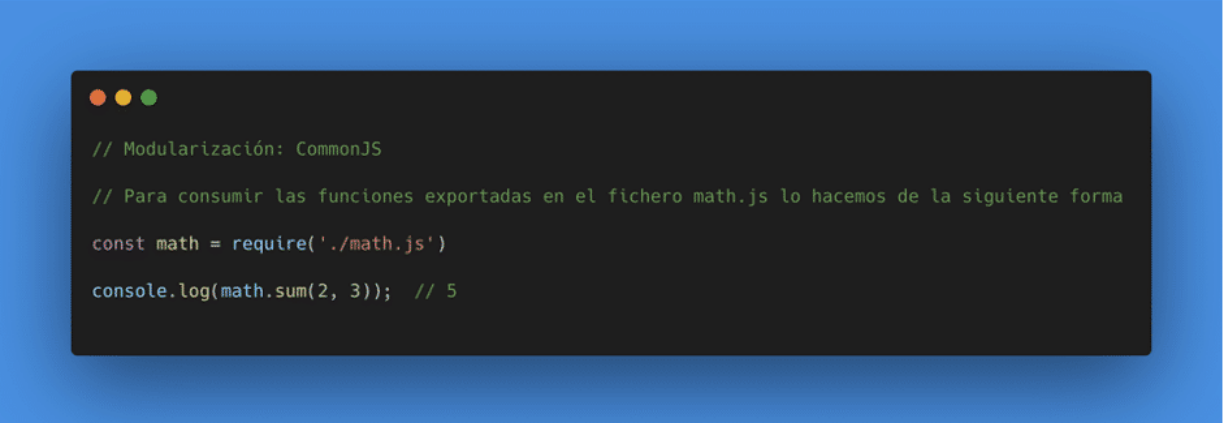
Usando el mismo código que vimos en los ESM, su equivalente en CommonJS sería este que muestro en la imagen.



Aquí vemos una palabra especial llamada **module**: el objeto representa lo que un módulo desea hacer disponible para otros módulos cuando son requeridos o importados.

La idea detrás de **module.exports** es proporcionar una forma de encapsular la funcionalidad privada dentro de un módulo y solo exponer aquello que se desee compartir con el mundo exterior (otros módulos).

Esta es la forma de consumir las funciones exportadas.



```
// Modularización: CommonJS  
  
// Para consumir las funciones exportadas en el fichero math.js lo hacemos de la siguiente forma  
  
const math = require('./math.js')  
  
console.log(math.sum(2, 3)); // 5
```

Si bien CommonJS es conocido principalmente por su adopción en Node.js, es importante mencionar que no está limitado solo a Node. Hay herramientas y bibliotecas, como Browserify y Webpack, que permiten usar CommonJS en el navegador al empaquetar y convertir los módulos CommonJS en código que puede ser ejecutado en el navegador.

AMD (*Asynchronous Module Definition*)

Es un sistema de módulos diseñado específicamente para el navegador con el objetivo de **cargar módulos de manera asíncrona**. Fue creado para resolver problemas de rendimiento en el navegador causados por la carga síncrona y bloqueante de módulos.

La biblioteca más popular que implementa AMD es **RequireJS**. A continuación, te mostramos un ejemplo de cómo se define y utiliza un módulo con AMD usando [RequireJS](#).

```
// Modularización: AMD

// Para definir un modulo usando RequireJS

// Entendemos que estan siendo exportadas desde un fichero llamado math.js

define([], function() { // define es una función definida en RequireJS
  return {
    sum: function(x, y) {
      return x + y;
    },
    multiply: function(x, y) {
      return x * y;
    },
    PI: 3.14.159
  }
})
```

Veamos qué representa cada parte del código mostrado:

- **define:** función proporcionada por RequireJS para definir un módulo.
- **[]:** primer argumento para *define* y representa las dependencias del módulo. En este caso, el array está vacío, lo que significa que el módulo *math.js* no tiene dependencias.
- **function() {...}:** función es el segundo argumento para *define*. Se invocará una vez que todas las dependencias (si existen) se hayan cargado. Lo que devuelva esta función (usando *return*) será el valor exportado del módulo.
- **return {...}:** estamos devolviendo un objeto con las funciones **sum**, **multiply** y la propiedad **PI** para ser consumidos. Este objeto será el valor del módulo cuando se requiera o importe en otro lugar.

Así se usa el módulo que hemos definido:



```
// Modularización: AMD

// Para importar un modulo usando RequireJS

define(['math'], function(math) {
    console.log(math.suma(2, 3)); // 5
});
```

Si interpretamos el código para importar, veremos que usamos nuevamente *define*, pero esta vez el array de dependencias contiene el fichero *math*; en este caso, al encontrarse dentro del mismo directorio, no necesitamos añadir la ruta relativa para encontrarlo. La función que recibe como segundo argumento a *define* ahora contiene el objeto con las funciones de *math.js* y es usado en la definición de esta.

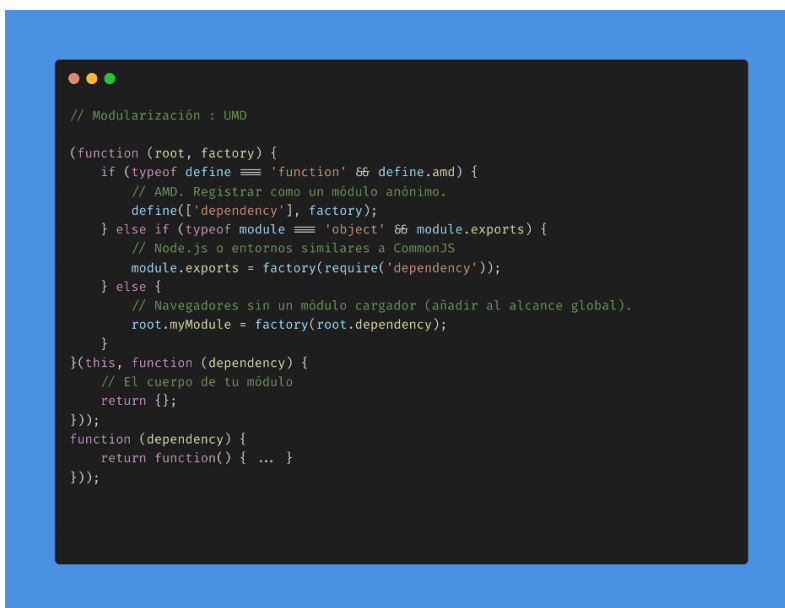
AMD proporcionó una solución sólida para la carga asíncrona de módulos mucho antes de la introducción y adopción generalizada de los módulos ES6. Sin embargo, con las herramientas modernas y el creciente soporte de ESM, su uso ha disminuido con el tiempo. Aun así, es un concepto importante que ha influido en el desarrollo web moderno.

UMD (*Universal Module Definition*)

Debido a que JavaScript ha evolucionado con diferentes sistemas de módulos que se utilizan en diferentes entornos, los desarrolladores de bibliotecas se encontraron en una situación en la que tenían que decidir para qué sistema de módulos escribirían su biblioteca, o incluso tener que proporcionar múltiples versiones para cada sistema de módulos.

Para solucionar este problema, desarrollaron un patrón que permite que un módulo de JavaScript funcione tanto con CommonJS (por ejemplo, en entornos Node.js) como con AMD (*Asynchronous Module Definition*, utilizado en navegadores con bibliotecas como RequireJS). Además, UMD también garantiza la compatibilidad en situaciones donde ni CommonJS ni AMD están presentes, permitiendo que el módulo se añada directamente al alcance global.

La idea detrás de UMD es simple: **proporcionar una solución universal para que los módulos se utilicen en cualquier entorno.**



```
// Modularización : UMD

(function (root, factory) {
  if (typeof define === 'function' && define.amd) {
    // AMD. Registrar como un módulo anónimo.
    define(['dependency'], factory);
  } else if (typeof module === 'object' && module.exports) {
    // Node.js o entornos similares a CommonJS
    module.exports = factory(require('dependency'));
  } else {
    // Navegadores sin un módulo cargador (añadir al alcance global).
    root.myModule = factory(root.dependency);
  }
})(this, function (dependency) {
  // El cuerpo de tu módulo
  return {};
});

function (dependency) {
  return function() { ... }
});
```

En este ejemplo podemos ver cómo el parámetro **root** pasado en la IIFE es una referencia al objeto global (*window* en navegadores, *global* en Node.js, etc.), mientras que **factory** es una función que define y devuelve tu módulo. Esta función toma cualquier dependencia que el módulo pueda tener, como sus argumentos.

¿Qué hemos aprendido?



En nuestra travesía por este fastbook, hemos destapado los rincones más profundos del lenguaje JavaScript, explorando las herramientas y técnicas que hacen de este lenguaje uno de los más versátiles y potentes del mundo del desarrollo web.

Algunos de los puntos principales que hemos aprendido:



Hemos profundizado en la esencia de los **callbacks**, esos métodos que nos permiten hacer un seguimiento de operaciones asíncronas. El **event loop** como proceso que gestiona el orden de ejecución del código junto con el **call stack**, **callback queue** y las Web APIs. ¡Y las **clausuras**!, estructuras que nos brindan la capacidad de mantener vivo el estado, incluso después de que una función ha finalizado su ejecución.



La **gestión de errores** y el empleo adecuado del objeto error son cruciales. En este mundo digital, donde el mínimo fallo puede tener consecuencias inimaginables, es vital estar preparado y saber cómo manejar los imprevistos.



Aprendimos cómo crear una **promesa** y exploramos los métodos útiles que nos ayudan a trabajar con ellas de manera eficiente.



O que **async/await** es, en esencia, una manera poética de decir: 'tómalo con calma, paso a paso'.



Adentrándonos en la identidad de las funciones, descubrimos el uso de **this**, **call**, **apply** y **bind**. Estas herramientas nos permiten definir y manipular el contexto en el que una función opera, otorgándonos un control preciso sobre su comportamiento.



La **modularización y encapsulación con IIFE** son conceptos que, aunque pueden parecer técnicos, son como los cimientos y los muros de una casa, esenciales para su estructura y protección.



Y no podríamos olvidar los **sistemas de módulos** que exploramos: ESM, CommonJS, AMD y UMD. Cada uno de estos tiene su lugar y un propósito, al igual que las herramientas en la caja de un artesano.

Por último, si sientes que hay más por descubrir, te animo a que te sumerjas en la vasta documentación y recursos disponibles. El aprendizaje nunca termina, y cada día es una oportunidad para fortalecer y expandir tus habilidades.

¡Enhorabuena! Fastbook superado